```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define FILE_NAME "tokens.csv"
#define NAME_SIZE 50
#define SERVICE_SIZE 30
#define STATUS_SIZE 20

typedef struct Token
{
  int tokenNumber;
  char customerName[NAME_SIZE];
  char service[SERVICE_SIZE];
  char status[STATUS_SIZE];
  struct Token *next;
} Token;

Token *allTokens = NULL;
Token *tail = NULL;
int nextTokenNumber = 1;

/* Function declarations */
void loadTokens(void);
int saveTokens(void);
void addToken(void);
void displayWaitingQueue(void);
void callNextToken(void);
void completeService(void);
void cancelToken(void);
void checkTokenStatus(void);
void displayAllTokens(void);
void cleanup(void);
void clearInput(void);
Token *findToken(int number);

/* Clear invalid input */
void clearInput(void)
{
  int ch;

  while ((ch = getchar()) != '\n' && ch != EOF)
```

```
    ;
}

/* Find a token using its number */
Token *findToken(int number)
{
  Token *current = allTokens;

  while (current != NULL)
  {
    if (current->tokenNumber == number)
      return current;

    current = current->next;
  }

  return NULL;
}

/* Save all token records to CSV */
int saveTokens(void)
{
  FILE *file = fopen(FILE_NAME, "w");

  if (file == NULL)
  {
    printf("ERROR: Could not open %s for saving.\n", FILE_NAME);
    return 0;
  }

  fprintf(file, "TokenNumber,CustomerName,Service,Status\n");

  Token *current = allTokens;

  while (current != NULL)
  {
    if (fprintf(file, "%d,%s,%s,%s\n",
                current->tokenNumber,
                current->customerName,
                current->service,
                current->status) < 0)
    {
      fclose(file);
```

```
      printf("ERROR: Could not write token records.\n");
      return 0;
    }

    current = current->next;
  }

  if (fclose(file) != 0)
  {
    printf("ERROR: Could not finish saving the CSV file.\n");
    return 0;
  }

  return 1;
}

/* Load previously saved records from CSV */
void loadTokens(void)
{
  FILE *file = fopen(FILE_NAME, "r");

  if (file == NULL)
  {
    /* Create the file if it does not exist */
    if (saveTokens())
      printf("New storage file created: %s\n", FILE_NAME);
    else
      printf("Warning: Storage file could not be created.\n");

    return;
  }

  char line[256];
  int loadedCount = 0;

  while (fgets(line, sizeof(line), file) != NULL)
  {
    Token *newNode = malloc(sizeof(Token));

    if (newNode == NULL)
    {
      printf("Warning: Not enough memory to load all records.\n");
      break;
```

```
    }

    int result = sscanf(
        line,
        "%d,%49[^,],%29[^,],%19[^\r\n]",
        &newNode->tokenNumber,
        newNode->customerName,
        newNode->service,
        newNode->status);

    if (result != 4)
    {
      free(newNode);
      continue; /* Skip the CSV header or invalid lines */
    }

    newNode->next = NULL;

    if (allTokens == NULL)
    {
      allTokens = newNode;
      tail = newNode;
    }
    else
    {
      tail->next = newNode;
      tail = newNode;
    }

    if (newNode->tokenNumber >= nextTokenNumber)
      nextTokenNumber = newNode->tokenNumber + 1;

    loadedCount++;
  }

  fclose(file);

  printf("%d token record(s) loaded from %s.\n",
         loadedCount, FILE_NAME);
}

/* Generate a new token */
void addToken(void)
```

```c
{
  Token *newNode = malloc(sizeof(Token));

  if (newNode == NULL)
  {
    printf("Memory allocation failed!\n");
    return;
  }

  newNode->tokenNumber = nextTokenNumber;

  printf("Enter customer name: ");

  if (scanf(" %49[^\n]", newNode->customerName) != 1)
  {
    printf("Invalid customer name.\n");
    free(newNode);
    clearInput();
    return;
  }

  /* Keep customer names compatible with simple CSV storage */
  for (int i = 0; newNode->customerName[i] != '\0'; i++)
  {
    if (newNode->customerName[i] == ',')
      newNode->customerName[i] = ' ';
  }

  int serviceChoice;

  printf("\nSelect Banking Service:\n");
  printf("1. Cash Deposit\n");
  printf("2. Cash Withdrawal\n");
  printf("3. Passbook Update\n");
  printf("4. Account Enquiry\n");
  printf("Enter service choice: ");

  if (scanf("%d", &serviceChoice) != 1)
  {
    printf("Invalid service choice.\n");
    clearInput();
    free(newNode);
    return;
```

```
}

switch (serviceChoice)
{
case 1:
  strcpy(newNode->service, "Cash Deposit");
  break;

case 2:
  strcpy(newNode->service, "Cash Withdrawal");
  break;

case 3:
  strcpy(newNode->service, "Passbook Update");
  break;

case 4:
  strcpy(newNode->service, "Account Enquiry");
  break;

default:
  printf("Invalid service. Token not created.\n");
  free(newNode);
  return;
}

strcpy(newNode->status, "Waiting");
newNode->next = NULL;

/* Append token to the records list */
if (allTokens == NULL)
{
  allTokens = newNode;
  tail = newNode;
}
else
{
  tail->next = newNode;
  tail = newNode;
}

nextTokenNumber++;
```

```
  printf("\nToken %d generated successfully!\n",
         newNode->tokenNumber);
  printf("Customer: %s\n", newNode->customerName);
  printf("Service: %s\n", newNode->service);

  if (saveTokens())
    printf("Token saved permanently to %s.\n", FILE_NAME);
  else
    printf("WARNING: Token is in memory but may not be saved.\n");
}

/* Display tokens that are waiting, in FIFO order */
void displayWaitingQueue(void)
{
  Token *current = allTokens;
  int count = 0;

  printf("\n========== WAITING QUEUE ==========\n");

  while (current != NULL)
  {
    if (strcmp(current->status, "Waiting") == 0)
    {
      printf("Token: %d | Customer: %s | Service: %s\n",
             current->tokenNumber,
             current->customerName,
             current->service);

      count++;
    }

    current = current->next;
  }

  if (count == 0)
    printf("No customers are waiting.\n");
}

/* Call the earliest waiting customer */
void callNextToken(void)
{
  Token *current = allTokens;
```

```c
  while (current != NULL &&
         strcmp(current->status, "Waiting") != 0)
  {
    current = current->next;
  }

  if (current == NULL)
  {
    printf("\nNo customers are waiting.\n");
    return;
  }

  strcpy(current->status, "In Progress");

  printf("\nNow serving Token %d\n", current->tokenNumber);
  printf("Customer: %s\n", current->customerName);
  printf("Service: %s\n", current->service);

  if (saveTokens())
    printf("Token status saved.\n");
}

/* Complete a service */
void completeService(void)
{
  int number;

  printf("Enter token number to complete: ");

  if (scanf("%d", &number) != 1)
  {
    printf("Invalid input.\n");
    clearInput();
    return;
  }

  Token *record = findToken(number);

  if (record == NULL)
  {
    printf("Token not found.\n");
  }
  else if (strcmp(record->status, "In Progress") != 0)
```

```
  {
    printf("Only an In Progress token can be completed.\n");
  }
  else
  {
    strcpy(record->status, "Completed");

    if (saveTokens())
      printf("Token %d completed and saved.\n", number);
  }
}

/* Cancel a waiting token */
void cancelToken(void)
{
  int number;

  printf("Enter token number to cancel: ");

  if (scanf("%d", &number) != 1)
  {
    printf("Invalid input.\n");
    clearInput();
    return;
  }

  Token *record = findToken(number);

  if (record == NULL)
  {
    printf("Token not found.\n");
  }
  else if (strcmp(record->status, "Waiting") != 0)
  {
    printf("Only a Waiting token can be cancelled.\n");
  }
  else
  {
    strcpy(record->status, "Cancelled");

    if (saveTokens())
      printf("Token %d cancelled and saved.\n", number);
  }
```

```c
}

/* Check an individual token's status */
void checkTokenStatus(void)
{
  int number;

  printf("Enter token number: ");

  if (scanf("%d", &number) != 1)
  {
    printf("Invalid input.\n");
    clearInput();
    return;
  }

  Token *record = findToken(number);

  if (record == NULL)
  {
    printf("Token not found.\n");
    return;
  }

  printf("\nToken Number: %d\n", record->tokenNumber);
  printf("Customer: %s\n", record->customerName);
  printf("Service: %s\n", record->service);
  printf("Status: %s\n", record->status);
}

/* Display every stored record */
void displayAllTokens(void)
{
  Token *current = allTokens;

  if (current == NULL)
  {
    printf("\nNo token records available.\n");
    return;
  }

  printf("\n================ ALL TOKEN RECORDS ================\n");
```

```c
  while (current != NULL)
  {
    printf("Token: %d | Customer: %s | Service: %s | Status: %s\n",
           current->tokenNumber,
           current->customerName,
           current->service,
           current->status);

    current = current->next;
  }
}

/* Free allocated memory before exiting */
void cleanup(void)
{
  while (allTokens != NULL)
  {
    Token *temp = allTokens;
    allTokens = allTokens->next;
    free(temp);
  }

  tail = NULL;
}

int main(void)
{
  int choice;

  printf("Loading saved bank token records...\n");
  loadTokens();

  do
  {
    printf("\n====================================\n");
    printf("      SMART BANK TOKEN SYSTEM\n");
    printf("====================================\n");
    printf("1. Generate New Token\n");
    printf("2. View Waiting Queue\n");
    printf("3. Call Next Customer\n");
    printf("4. Complete Service\n");
    printf("5. Cancel Waiting Token\n");
    printf("6. Check Token Status\n");
```

```
    printf("7. View All Token Records\n");
    printf("0. Save and Exit\n");
    printf("-------------------------------------\n");
    printf("Enter your choice: ");

    if (scanf("%d", &choice) != 1)
    {
      printf("Please enter a valid number.\n");
      clearInput();
      continue;
    }

    switch (choice)
    {
    case 1:
      addToken();
      break;

    case 2:
      displayWaitingQueue();
      break;

    case 3:
      callNextToken();
      break;

    case 4:
      completeService();
      break;

    case 5:
      cancelToken();
      break;

    case 6:
      checkTokenStatus();
      break;

    case 7:
      displayAllTokens();
      break;

    case 0:
```

```
      if (saveTokens())
        printf("All records saved successfully.\n");

      printf("Thank you for using Smart Bank Token System!\n");
      break;

    default:
      printf("Invalid choice. Please try again.\n");
    }

  } while (choice != 0);

  cleanup();
  return 0;
}
```